

沈阳航空航天大学
计算机学院

实验报告

课程名称

操作系统

专 业

物联网工程

班 级

2234280102

学 号

223428010210

学生姓名

陈梓欣

指导教师

宗传玉

实验时间

周五 9-10 节

实验地点

工训楼 310

一、实验名称

系统调用

二、实验目的

1. 掌握 Linux 系统调用工作原理和 Linux 系统调用的执行过程。
2. 掌握增加系统调用及添加内核函数的编程方法。

三、实验内容和要求

1. 在 Linux0.11 内核中添加一个新的系统调用函数 max 函数，函数原型：

`int max(int a, int b, int c)`

该函数实现比较三个参数的大小，并将最大值作为函数返回值；

2. 在 Linux 0.11 内核中添加两个系统调用函数 Iam 和 Whoami，函数原型如下：

(1) `int Iam(const char* name)`；将字符串 name 的内容保存到内核中，返回值是拷贝的字符数，如果 name 长度大于 32，则返回-1，并置 errno 为 EINVAL。

(2) `int Whoami(char* name, int size)`；将 Iam 保存到内核中的字符串拷贝到数据缓冲区 name 中，size 为数据缓冲区 name 的长度，返回值是拷贝的字符数。如果 size 小于所需空间，则返回-1，并置 errno 为 EINVAL。

编写两个应用程序。其中，Iam 应用程序调用 Iam 函数，并使用命令行中的第一个参数作为 Iam 函数的参数；Whoami 应用程序调用 Whoami 函数，并打印输出获取的字符串。测试效果如下图：

```
[/usr/root]# Iam 123
[/usr/root]# Whoami
123
[/usr/root]#
```

图 1 测试结果

四、实验设计

系统调用是操作系统为应用程序提供的与内核进行交互的一组接口。通过这些接口，用户态应用程序的进程可以切换到内核态，由系统调用对应的内核函数代表该进程继续运行，从而可以访问操作系统维护的各种资源，实现应用程序与内核的交互。

系统调用通过中断机制向内核提交请求，系统调用的功能由内核函数实现，进入内核后不同的系统调用会有其各自对应的内核函数，这些内核函数就是系统调用的“服务例程”。

Linux 内核为了区分不同的系统调用，为每个系统调用分配了唯一的系统调用号。这些系统调用号定义在文件 `include/unistd.h` 中，系统调用号实际上对应于 `include/linux/sys.h` 中定义的系统调用函数指针表数组 `sys_call_table[]` 中项的索引值（也就是数组的下标）。对于所有系统调用的实现函数，内核把它们按照系统调用功能号顺序排列成一张函数指针表（在 `include/linux/sys.h` 文件中）。然后在中断 `int0x80` 的处理过程中，根据用户提供的功能号调用对应系统调用函数进行处理。

在 Linux 中规定 `int 0x80` 指令是系统调用的总入口，系统调用号存放在 `EAX` 寄存器中，应用程序通过系统调用传递给内核函数的参数依次存放于 `EBX`、`ECX` 和 `EDX` 这三个寄存器中（即系统调用最多只能有 3 个参数）。

Linux 系统调用的执行过程为：首先，应用程序准备好系统调用需要的参数，并直接或间接（通过库函数）发出一个调用请求，即调用 `int 0x80` 指令；然后，该指令通过陷阱处理机制（中断处理机制的一种），使该系统调用进入内核的入口点 `_system_call` 函数（`kernel/system_call.s` 文件中第 101 行）；最后，由 `_system_call` 函数找到系统调用对应的内核函数，该内核函数将被执行并通过 `EAX` 寄存器返回结果。系统调用处理过程的整个流程见下图。

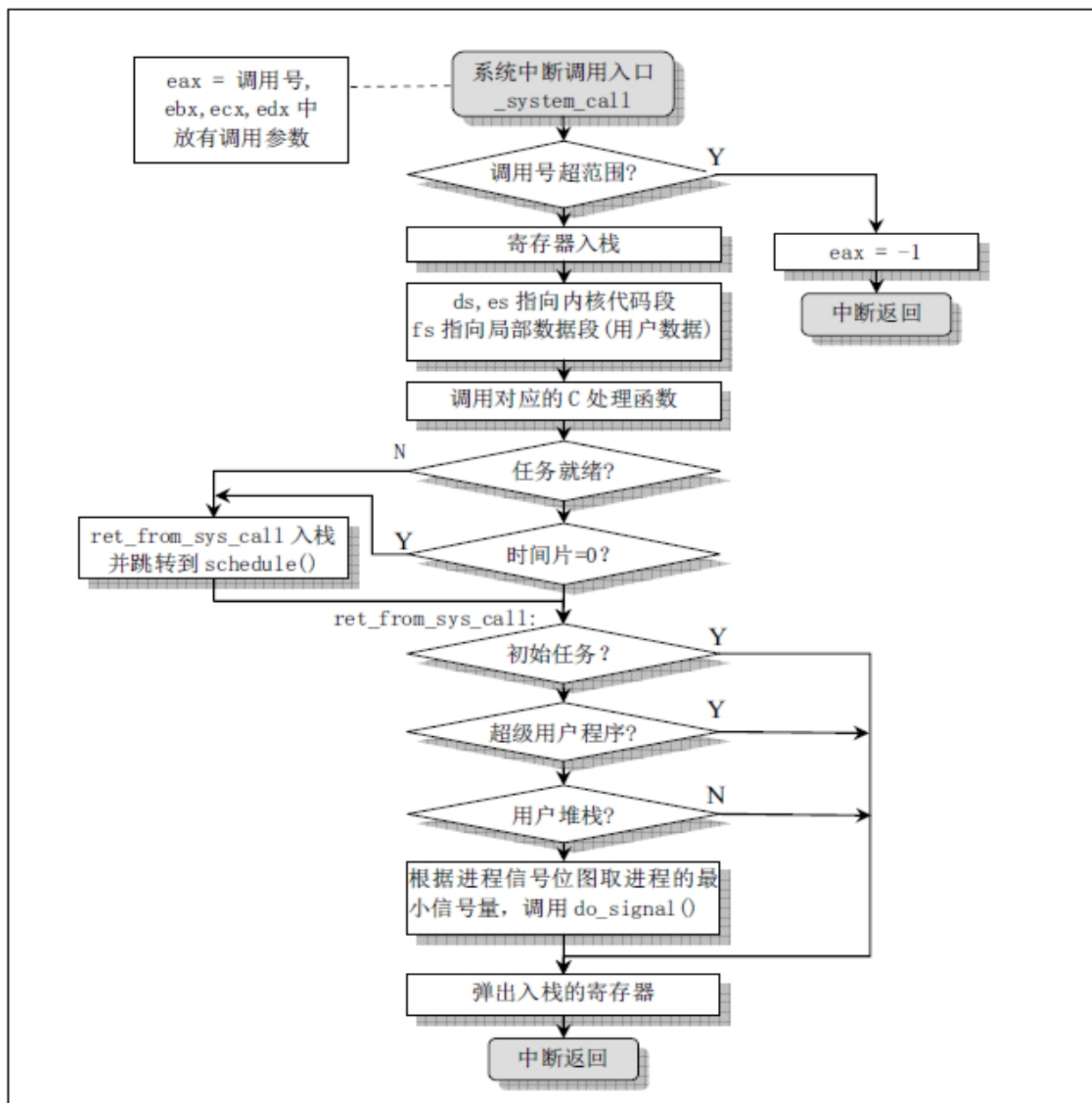


图 2 系统调用流程图

五、实验步骤及实验结果

1. 准备实验

- 1) 启动 Engintime Linux Lab。
- 2) 新建一个 Linux011 Kernel 实验项目。

2. 添加新的系统调用函数

为 Linux 0.11 添加 5 个新的系统调用函数，分别是：

- ① maxALL——该函数实现比较三个参数的大小并将较大值返回的功能。
- ② Iam——将输入的字符串内容保存到内核中,返回值是拷贝的字符数(限

制字符串长度最长为 32)。

③ Whoami: 将 Iam 保存到内核中的字符串拷贝到数据缓冲区中 (限制字符串长度最长为 32)。

④ TIam——将输入的字符串内容保存到内核中, 返回值是拷贝的字符数 (限制字符串长度最长为 12)。

⑤ TWhoami——将 Iam 保存到内核中的字符串拷贝到数据缓冲区中 (限制字符串长度最长为 12)。

3. 步骤如下:

1) 首先要为新系统调用分配一个唯一的系统调用号, 以原来的最大系统调用号为基础加 1 作为新的系统调用号。在“项目管理器”窗口中双击打开 include/unistd.h 文件, 在第 163-167 行添加新的系统调用号, 如图 2 所示:

```
163 #define __NR_maxALL 88
164 #define __NR_Iam 89
165 #define __NR_Whoami 90
166 #define __NR_TIam 91
167 #define __NR_TWhoami 92
```

图 3 添加新的系统调用号

2) 添加新系统调用号的同时, 也要使系统调用总数在原来的基础上增加 1。打开 kernel/system_calls 文件, 修改在第 73 行定义的系统调用总数, 如下图:

```
73 nr_system_calls = 93 # Linux 0.11 版内核中的系统调用总数。
```

图 4 修改 Linux 内核的系统调用总数

3) 在 include/linux/sys.h 文件中的第 90-98 行使用 C 语言声明内核函数的原型, 如图 4 所示。注意, 这里定义的函数原型并不需要与函数的定义完全一致, 只需要定义函数的返回值为 int 类型, 参数为空, 能够让标识符代表一个函数名称即可。

```
90 extern int sys_maxALL();
91
92 extern int sys_Iam();
93
94 extern int sys_Whoami();
95
96 extern int sys_TIam();
97
98 extern int sys_TWhoami();
```

图 5 使用 C 语言声明内核函数的原型

在此文件的最后，向系统调用函数指针表 `sys_call_table[]` 中添加新系统调用函数的指针（注意，系统调用号必须与系统调用内核函数指针在系统调用函数表中的索引一一对应），如图 5 所示：

```

191 | sys_maxALL,          //88
192 |
193 | sys_Iam,             //89
194 | sys_Whoami,          //90
195 |
196 | sys_TIam,            //91
197 | sys_TWwhoami         //92

```

图 6 在系统调用函数指针表中增加内核函数的指针

4) 在 `kernel/sys.c` 文件的最后编写代码，实现新系统调用对应的内核函数，如下图所示：

```

357 | int sys_maxALL(int a, int b, int c) {
358 |     int t;
359 |     t = a > b ? a : b;
360 |     return t > c ? t : c;
361 | }

```

图 7 系统调用内核函数 maxALL

```

363 | //char tname[32];
364 | char tname[32] = {0};
365 |
366 | int sys_Iam(const char* name)
367 | {
368 |     int i = 0;
369 |     int len = 0;
370 |     while(get_fs_byte(name + len) != '\0')
371 |         len++;
372 |     if(len > 32) {
373 |         errno = EINVAL;
374 |         return -1;
375 |     }
376 |
377 |     while(i < len)
378 |     {
379 |         tname[i] = get_fs_byte(name + i);
380 |         i++;
381 |     }
382 |     tname[i] = '\0';
383 |     return len;
384 | }

```

图 8 系统调用内核函数 Iam


```

386  int sys_Whoami(char* name, int size)
387  {
388      int i = 0;
389      int len = 0;
390      while(tname[len] != '\0')
391          len++;
392      if(size < len) {
393          errno = EINVAL;
394          return -1;
395      }
396
397      for(i = 0 ; i <= len ; i++)
398      {
399          put_fs_byte(tname[i], name + i);
400      }
401      return len;
402  }
403

```

图 9 系统调用内核函数 Whoami

```

404  char Ttname[12] = {0};
405
406  int sys_Tlame(const char* name)
407  {
408      int i = 0;
409      int len = 0;
410      for (i = 0; i < sizeof(Ttname); i++) {
411          Ttname[i] = 0; // Initialize to zero
412      }
413      while(get_fs_byte(name + len) != '\0')
414          len++;
415      if(len > 12) {
416          errno = ENAMETOOLONG;
417          return -1;
418      }
419      i = 0;
420      while(i < len)
421      {
422          Ttname[i] = get_fs_byte(name + i);
423          i++;
424      }
425      Ttname[i] = '\0';
426      return len;
427  }
428

```

图 10 系统调用内核函数 Tlame


```

429  int sys_TWhoami(char* name, int size)
430  {
431      int i = 0;
432      int len = 0;
433      while(Ttname[len] != '\0')
434          len++;
435      if(len == 0) {
436          return -1;
437          printf("-1\n");
438      }
439      else if(size < len) {
440          errno = -1;
441          return -1;
442      }
443      for(i = 0 ; i <= len ; i++)
444      {
445          put_fs_byte(Ttname[i], name + i);
446      }
447      return len;
448  }

```

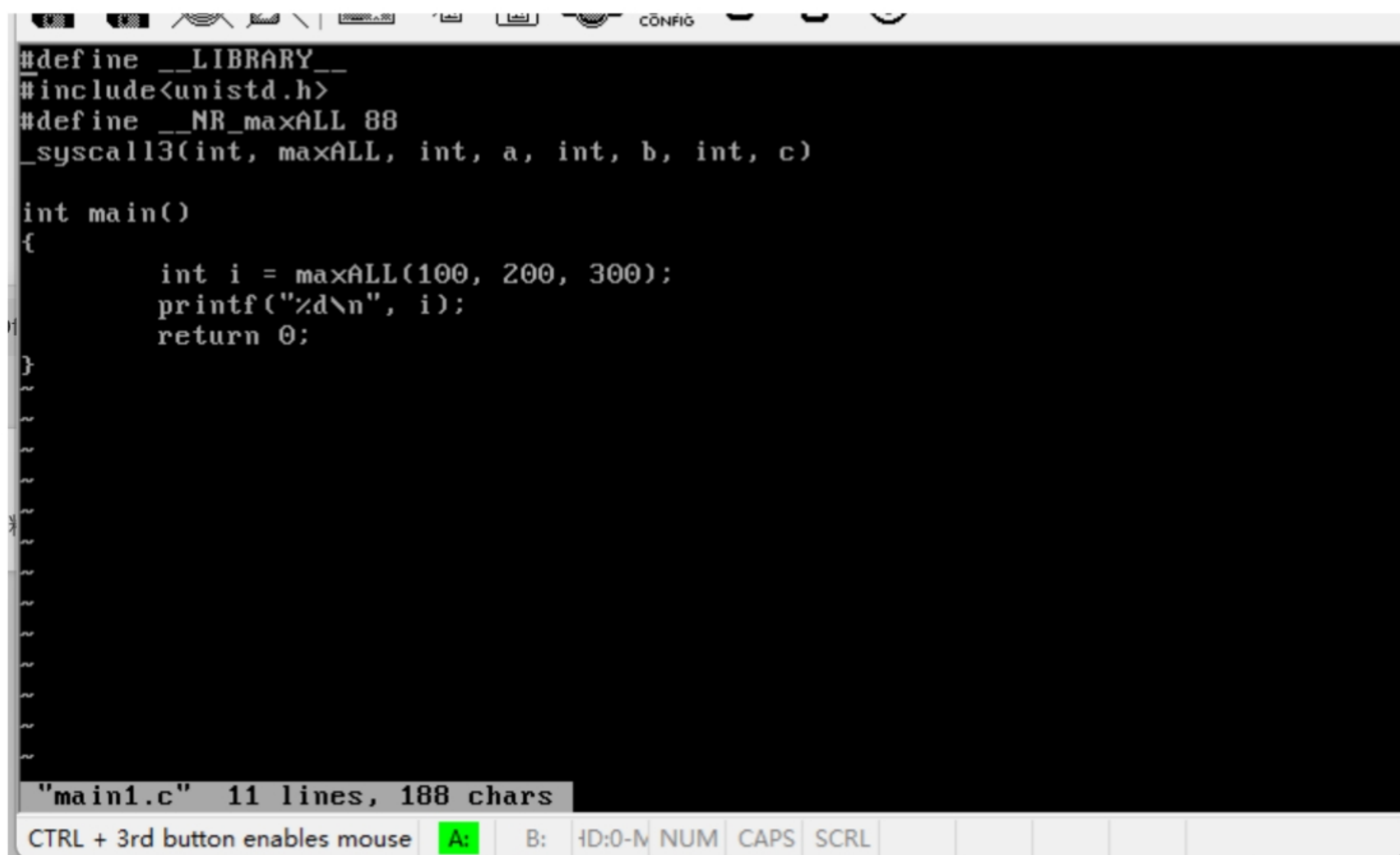
图 11 系统调用内核函数 TWhoami

5) 按 F7 生成 Linux 0.11 内核，修改语法错误直到生成成功。接下来需要在 Linux 0.11 中分别编写应用程序来测试新系统调用是否添加成功。

4. 在应用程序中测试新系统调用

1) 按 F5 启动调试。

2) 待 Linux 0.11 启动后，使用 vi 编辑器新建一个 main1.c 文件。编辑 main1.c 文件中的源代码（如图 7 所示）。其中，需要定义 `__LIBRARY__` 宏及包含 `unistd.h` 头文件，还需要再次定义 `__NR_max` 宏，并使用 `_syscall2` 宏对系统调用函数进行定义，当该宏展开时，就会在源代码文件中使用 C 语言添加 `maxALL` 函数的完整实现。



```
#define __LIBRARY__
#include<unistd.h>
#define __NR_maxALL 88
__syscall3(int, maxALL, int, a, int, b, int, c)

int main()
{
    int i = maxALL(100, 200, 300);
    printf("%d\n", i);
    return 0;
}
```

"main1.c" 11 lines, 188 chars

CTRL + 3rd button enables mouse A: B: ID:0-M NUM CAPS SCRL

图 12 编辑 main1.c 文件

3) 使用命令 `gcc main.c -o main` 生成可执行文件 main1。



```
[/usr/root]# gcc main1.c -o main1
[/usr/root]# main1
```

4) 执行 `sync` 命令，将文件保存到磁盘。



```
OK.
[/usr/root]# sync main1.c
[/usr/root]#

[/usr/root]# gcc main1.c -o main1
[/usr/root]# sync main1
[/usr/root]#
```

图 13 将文件保存到磁盘

5) 执行 `chmod +x main` 命令为 main 文件添加可执行权限。



```
[/usr/root]# chmod +x main1
[/usr/root]# ls -l main1
-rwx--x--x 1 root root 24707 ??? ?? ??? main1
[/usr/root]#
```

图 14 添加权限

6) 运行 main1。如果 main1 能输出正确的结果，则说明新系统调用添加成功。



```
[/usr/root]# main1
300
[/usr/root]#
```

图 15 正确返回最大值

5. 仿照上述 2) -6) 步骤，分别测试新系统调用 `Iam` 与 `Whoami`。


```
[/usr/root]# cat main2.c
#define __LIBRARY__
#include<unistd.h>
#include<stdio.h>
#define __NR_Iam 89

_syscall1(int, Iam, const char*, name)

int main(int argc, char** argv)
{
    int ans = Iam(argv[1]);
    return 0;
}
[/usr/root]#
```

图 16 编辑 main2.c 文件

```
[/usr/root]# cat main3.c
#define __LIBRARY__
#include<unistd.h>
#include<stdio.h>
#define __NR_Whoami 90

_syscall2(int, Whoami, char*, name, int, size)

int main(int argc, char** argv)
{
    char name[33] = {0};
    int ans = Whoami(name, 33);
    printf("%s\n", name);
    return 0;
}
[/usr/root]#
```

图 17 编辑 main3.c 文件

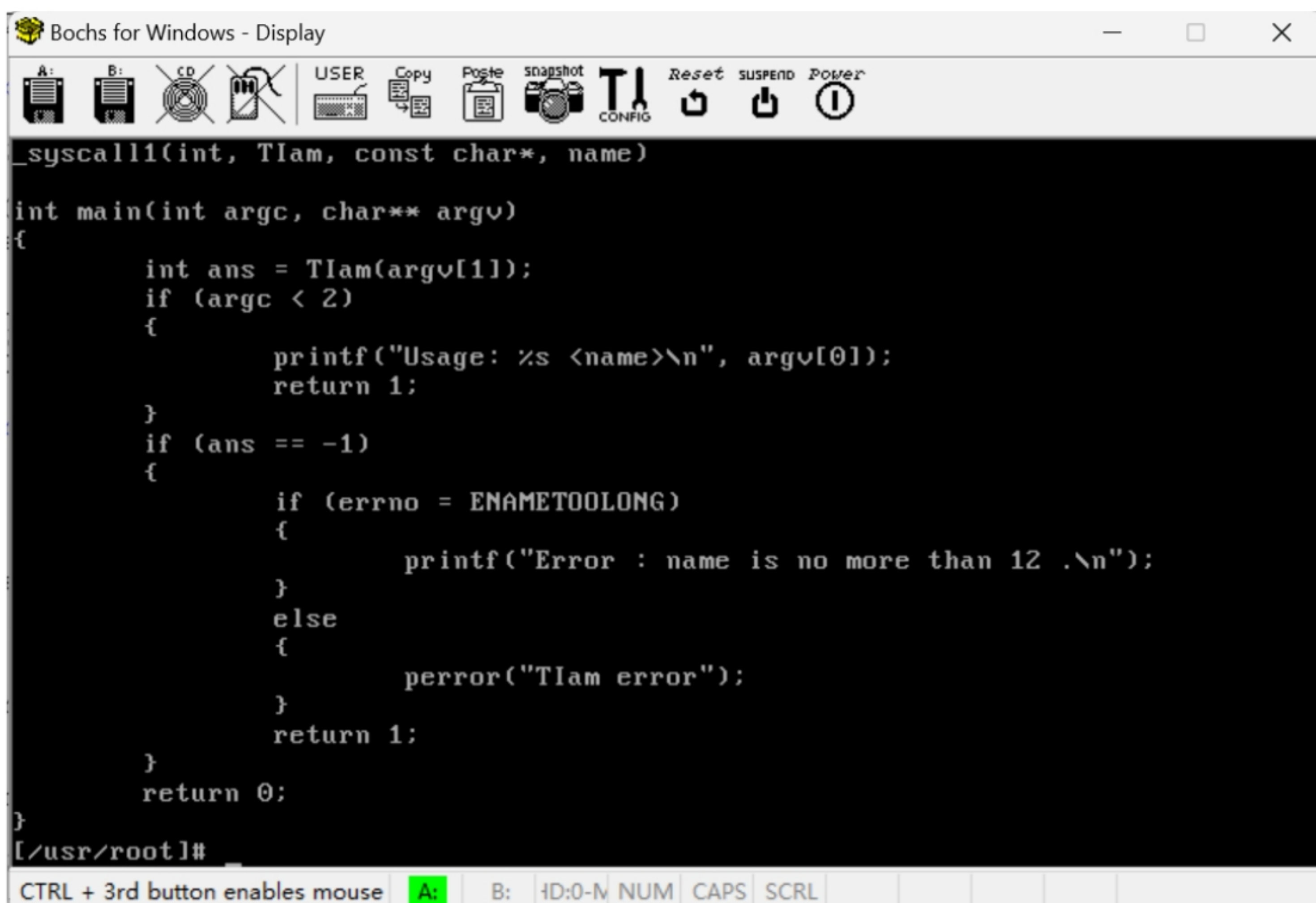
```
[/usr/root]# Iam 123
[/usr/root]# Whoami
123
[/usr/root]# Iam czx
[/usr/root]# Whoami
czx
[/usr/root]#
```

图 18 正确返回输入的字符串

6. 在 Linux 0.11 内核中添加两个系统调用函数 TIam 和 TWhoami。

要求：名字最大长度是 12，如果用户输入字符串长度大于 12，TIam 程序

提示：name is no more than 12。此时运行 TWhoami 程序，提示 -1



```

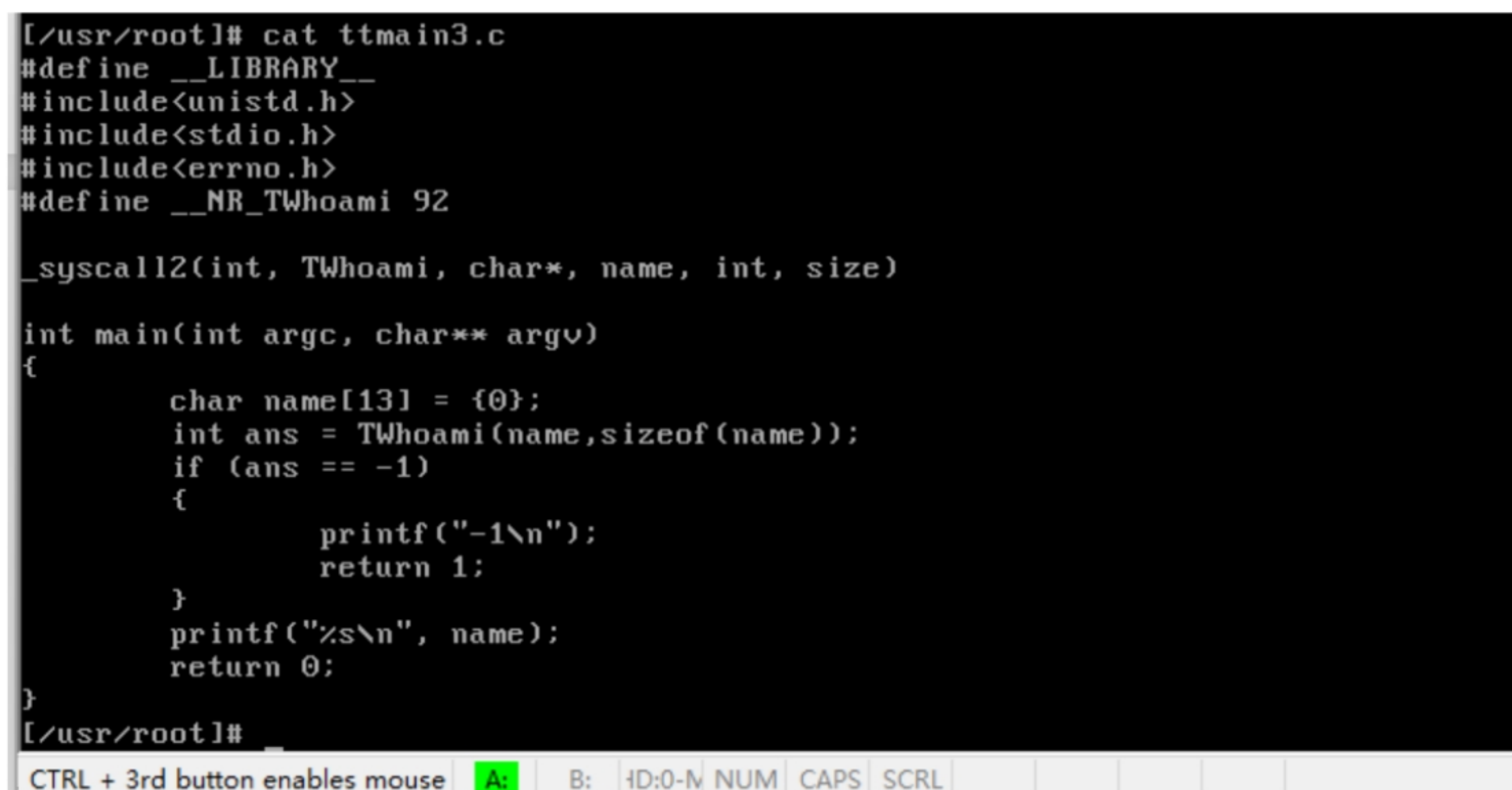
_syscall1(int, TlAm, const char*, name)

int main(int argc, char** argv)
{
    int ans = TlAm(argv[1]);
    if (argc < 2)
    {
        printf("Usage: %s <name>\n", argv[0]);
        return 1;
    }
    if (ans == -1)
    {
        if (errno = ENAMETOOLONG)
        {
            printf("Error : name is no more than 12 .\n");
        }
        else
        {
            perror("TlAm error");
        }
        return 1;
    }
    return 0;
}
[/usr/root]#

```

CTRL + 3rd button enables mouse A: B: HD:0-M NUM CAPS SCRL

图 19 编辑 ttmain2.c 文件



```

[/usr/root]# cat ttmain3.c
#define __LIBRARY__
#include<unistd.h>
#include<stdio.h>
#include<errno.h>
#define __NR_TWhoami 92

_syscall2(int, TWhoami, char*, name, int, size)

int main(int argc, char** argv)
{
    char name[13] = {0};
    int ans = TWhoami(name, sizeof(name));
    if (ans == -1)
    {
        printf("-1\n");
        return 1;
    }
    printf("%s\n", name);
    return 0;
}
[/usr/root]#

```

CTRL + 3rd button enables mouse A: B: HD:0-M NUM CAPS SCRL

图 20 编辑 ttmain3.c 文件



```

[/usr/root]# ttIam 123456
[/usr/root]# ttWhoami
123456
[/usr/root]# ttIam qazwert12
[/usr/root]# ttWhoami
qazwert12

```

图 21 输入字符串符合要求则正确返回


```
[/usr/root]# ttIam 1234567890111q
Error : name is no more than 12 .
[/usr/root]# ttWhoami
-1
[/usr/root]#
```

CTRL + 3rd button enables mouse A: B: ID:0-N NUM CAPS SCRL

图 22 字符串长度大于 12 则按要求报错

六、实验出现的问题及解决方法

问题：运行调用 TIam 函数编写的程序，当输入字符串长度大于 12 时，所输入的字符串无法被正确存入内核，则当前程序无法覆盖掉内核中已有的数据，导致接下来运行调用 TWhoami 函数的程序无法返回预期的-1。

解决方法：在 TWhoami 函数中添加一个 ofr 循环语句，实现：在每一次调用 TWhoami 函数从用户空间读取数据前，先清空上一次程序运行时的数据残留，避免当输入字符串长度大于 12 时误读残留数据导致程序无法正确返回-1。

七、结论

本次实验通过向 Linux 0.11 内核添加新的系统调用函数，深入理解了 Linux 系统调用的工作原理及其实现过程。实验首先通过学习 Linux 内核的系统调用机制，掌握了如何通过中断 (int 0x80) 进入内核，调用系统调用函数并传递参数。实验中，我们添加了一个`max`函数，该函数实现比较三个参数的大小并返回最大值。此外，还实现了两个新的系统调用：`Iam`用于将字符串保存到内核中，`Whoami`则从内核中读取该字符串并返回。

在实现过程中，我们熟悉了 Linux 内核中系统调用的注册与调度机制。通过修改内核源代码，如`unistd.h`、`sys.h`、`sys\_call\_table`等文件，成功为内核添加了新的系统调用。实验中还通过调试工具分析了系统调用执行的过程，掌握了如何通过寄存器传递参数和返回值，如何调试系统调用，并确保在内核中正确保存和读取数据。

通过本次实验，我对 Linux 内核的系统调用机制、内核函数的实现和调试过程有了更加深入的理解，也提高了在 Linux 环境下进行系统级编程和调试的能力。